

Image Upscale and Vectorization App - Product Plan and Technical Design

Prepared for: Product planning and app design

Date: 2026-05-04

Output formats: Markdown, DOCX, and PDF

Research basis: Public product pages, public API documentation, public app listings, and open-source project documentation. This report does not rely on reverse engineering or private implementation details.

1. Executive summary

The recommended product is a **web-first image enhancement platform** that can later support a mobile app and AI chat bot. The platform should accept a user image and output one or both of the following:

1. **Upscaled raster image:** higher-resolution PNG, JPEG, WebP, or TIFF.
2. **Vectorized artwork:** clean SVG first, then PDF, EPS, DXF, and high-resolution PNG previews.

The strongest market position is not simply “an upscaler” or “an SVG converter.” The best opportunity is a **single guided workflow for low-quality logos, icons, stickers, decals, scanned sketches, AI-generated artwork, and print assets:**

Upload poor-quality image -> auto-detect image type -> clean and upscale if needed
-> vectorize if appropriate -> preview -> edit simple controls -> export production-ready files.

The five reference apps suggest three clear product lessons:

- **Vectorizer.io** shows the value of image cleanup before vectorization and exposes practical vectorization controls such as model type, color count, antialiasing, detail level, and minimum shape area [S1][S2].
- **Vectorizer.AI** shows what high-end SVG conversion should feel like: automatic raster-to-vector conversion, interactive preview, palette control, transparency support, geometry-aware shape fitting, and many output compatibility options [S3][S4].
- **Topaz Labs and the Poe TopazLabs bot** show the importance of image-type-specific upscaling models, async job processing, cloud rendering, local/private rendering positioning, and simple conversational controls such as scale factor or output dimensions [S5][S6][S7][S8][S9][S10].
- **The mobile logo maker reference** is less about vectorization and more about mobile UX: users want to describe or import a logo, get fast options, customize colors/fonts/layout, and export shareable brand files [S11].

The MVP should be a **web app plus API backend**, not a mobile-only app. Web is faster to test, easier for file upload/download workflows, and better for SVG preview and editing. Mobile and AI bot versions should reuse the same backend.

2. Product vision

Product name placeholder

Use a working name such as **VectorLift**, **SharpVector**, or **LogoLift AI** during planning. The final name can be decided after brand testing.

One-sentence product promise

Turn blurry images, logos, and sketches into sharp high-resolution images and clean editable SVG files.

Main customer promise

The user should not need to know whether their file needs denoising, upscaling, background removal, color quantization, line tracing, curve fitting, or SVG optimization. The app should decide automatically and expose simple controls only when useful.

Recommended first platform

Build in this order:

1. **Web app** - core product, preview, payment, batch processing, API onboarding.
 2. **API** - required from the beginning because all future clients use it.
 3. **AI bot** - thin wrapper over the API, useful for Poe/Discord/Telegram/WhatsApp style flows.
 4. **Mobile app** - best after the backend and web UX are validated.
-

3. Target users and use cases

Primary users

- Small business owners with low-resolution logos.
- Designers who need quick SVG conversion for client files.
- Print shops, sticker shops, embroidery shops, screen printers, laser cutters, and CNC shops.
- Etsy and Shopify sellers making decals, shirts, labels, and packaging.
- AI-art creators who need sharper images for print or large screens.
- Marketing teams converting legacy PNG/JPEG assets into scalable brand files.

High-value use cases

1. **Logo rescue:** turn a blurry JPEG logo into SVG, transparent PNG, PDF, and social media sizes.
 2. **Sticker/decal prep:** convert flat artwork into clean vector paths for print/cut workflows.
 3. **Sketch to vector:** convert scanned black-and-white or colored sketches into SVG.
 4. **AI art upscaling:** upscale generated images to 2x or 4x with artifact cleanup.
 5. **Photo upscaling:** enlarge photos for print, web, restoration, and cropping.
 6. **Transparent PNG upscaling:** preserve alpha transparency and clean edge halos.
 7. **Text and shape enhancement:** sharpen screenshots, labels, scanned packaging, and old signs.
-

4. Competitive research summary

4.1 Vectorizer.io

Publicly visible positioning:

- Converts raster images such as PNG, BMP, and JPEG to SVG, EPS, and DXF [S1].
- Describes vectorization as converting pixel color information into geometric primitives such as lines, circles, and curves [S1].
- Offers an experimental **Enhance image with AI** option that removes noise, simplifies colors, and improves edges before vectorization [S1].
- Its API documentation shows a /v4.0/vectorize endpoint, credits-code authentication, and request controls such as format, colors, model, algorithm, detail level, antialiasing, minimum area, width, unit, configuration file, and palette import [S2].
- The API docs state that the API is no longer available to new users as of a March 2024 update, which is important if considering direct integration [S2].

Design lessons:

- Preprocessing before vectorization should be a first-class feature.
- Advanced users care about color count, model type, antialiasing, detail simplification, minimum shape area, and palette constraints.
- For API design, include explicit error handling, payload limits, and retry guidance.

4.2 Vectorizer.AI

Publicly visible positioning:

- Converts PNG, JPG, GIF, BMP, and WebP images into SVG, EPS, DXF, PDF, and PNG outputs [S3].
- Uses a simple flow: pick image, process, inspect full interactive preview, download [S3].
- Describes a combined system of deep learning networks and classical algorithms, a proprietary vector graph, full shape fitting, curve support, clean corners, symmetry

modeling, adaptive simplification, palette control, sub-pixel precision, alpha transparency, and export options [S3].

- States that it performs actual vectorization, not merely embedding a bitmap inside an SVG wrapper [S3].
- Public API docs show a vectorize endpoint, a download endpoint, a delete endpoint, account status, output options, authentication, rate limiting, timeouts, and response headers [S4].
- Public FAQ says current limits are 3 megapixels and 30 MB, and states a 24-hour retention policy for uploaded images and results [S3].

Design lessons:

- High-quality vectorization needs a geometry pipeline, not just AI segmentation.
- The preview-before-purchase model is powerful because users can judge SVG quality visually.
- The product should support true SVG, not embedded bitmap SVG.
- Compatibility options matter: Illustrator, Cricut, CAD, print, cutting, and embroidery workflows have different tolerances.

4.3 Topaz Labs

Publicly visible positioning:

- Topaz Gigapixel positions itself around AI image upscaling and the idea of adding the “right pixels,” not just enlarging pixels [S5].
- Gigapixel documentation supports scale factor up to 6x or custom dimensions, with a 32,000 px longest-side maximum [S6].
- Topaz developer documentation groups Gigapixel as precision upscaling for images, with models such as Standard 2, High Fidelity 2, Low Resolution 2, Art & CGI, Transparent Image Upscale, Detail Faces, Recover Faces, and Text & Shapes [S7].
- Topaz API documentation shows async processing patterns: submit a job, receive process_id and eta, then use status and download endpoints [S8][S9].
- Topaz API docs also show webhook support and exponential-backoff retries for webhook delivery [S8].

Design lessons:

- Use different upscaling presets for photos, faces, low-res files, art/CGI, transparent PNGs, and text/shapes.
- Async processing should be standard. Large image tasks should not block the browser request.
- The backend should return previews early and final downloads when processing completes.

4.4 Poe TopazLabs bot

Publicly visible positioning:

- The Poe TopazLabs bot accepts image input and exposes optional parameters such as upscale factor up to 16, output height, output width, and whether the input is AI-generated [S10].
- With no parameters, the bot defaults to increasing input height and width by 2x, and it is positioned as especially effective on human faces [S10].

Design lessons:

- A chat bot can be a thin conversational interface over the same backend API.
- Defaults matter. Most users should be able to upload an image and say “make this sharp” or “make this SVG.”
- Parameters should be optional and human-readable.

4.5 Mobile AI Logo Maker reference

Publicly visible positioning:

- The referenced Google Play package is listed by AppBrain as **AI Logo Maker** by FIRE Ltd, an Art & Design app with 100,000+ downloads, an Android version listing, rating/review counts, and an app description around creating professional logos quickly [S11].
- The listing language emphasizes ease, AI-powered logo creation, customization, and high-quality export [S11].

Design lessons:

- Mobile users respond to guided tasks, not technical labels.
 - For the mobile version, use simple actions: “Sharpen,” “Make SVG,” “Remove background,” “Create logo kit,” and “Export for print.”
 - The mobile app should include simple brand-kit exports, not a full professional vector editor.
-

5. Product strategy

Recommended positioning

Position the product as:

AI-powered image upscaling and SVG vectorization for logos, artwork, photos, and print-ready assets.

Avoid positioning only as a logo maker or only as an upscaler. The strongest wedge is **repairing and preparing existing images**, especially logos and art files that users already have.

Differentiators

1. **One-click auto route:** detect whether the image is a photo, logo, transparent PNG, text screenshot, sketch, AI artwork, or scan.
2. **Upscale + vectorize combo:** improve the raster input before tracing it into SVG.

3. **True SVG guarantee:** output contains editable vector paths, not an embedded raster image.
4. **Simple editor:** palette edit, background removal, speckle removal, smoothing, and path simplification.
5. **Production exports:** SVG, PDF, EPS, DXF, transparent PNG, social sizes, favicon, app icon sizes.
6. **API and batch mode:** important for print shops and marketplaces.
7. **Privacy option:** short retention by default; enterprise option for no training and configurable deletion.

Product modes

The app should expose three top-level modes:

1. **Upscale Image**
Best for photos, AI art, portraits, product images, and raster assets.
 2. **Vectorize Image**
Best for logos, icons, stickers, sketches, decals, line art, flat illustrations, and scanned drawings.
 3. **Fix Logo / Enhance + Vectorize**
Best for low-resolution logos and artwork that need cleanup before SVG export.
-

6. User experience design

6.1 Web app core flow

1. User lands on homepage.
2. User drags an image onto the upload area or pastes from clipboard.
3. App instantly creates a small preview and runs image classification.
4. App recommends one action:
 - “Upscale this photo”
 - “Make this logo SVG”
 - “Clean and vectorize this artwork”
 - “Remove background and export logo kit”
5. User can accept auto mode or open advanced controls.
6. Backend creates async job.
7. UI shows progress and an early preview.
8. User compares before/after with zoom and slider.
9. User pays or uses credits to unlock full download.
10. User downloads selected formats or a ZIP package.

6.2 Upload screen

Primary UI copy:

Drop an image here to upscale or convert it to SVG.

Secondary actions:

- Paste image
- Import from URL
- Batch upload
- Try sample image

Supported input labels:

- PNG
- JPG/JPEG
- WebP
- BMP
- GIF first frame
- HEIC later for mobile
- SVG cleanup later

6.3 Preview screen

Preview controls:

- Before/after slider
- Zoom: 100%, 200%, 400%, fit
- Transparent checkerboard toggle
- SVG path preview toggle
- Pixel preview vs vector preview
- Difference overlay for pro users
- File-size and path-count indicators

6.4 Advanced vector controls

Keep advanced settings hidden by default:

- Max colors: auto, 2, 4, 8, 16, 32
- Smoothness: low, medium, high
- Speckle removal: off, low, medium, high
- Edge sharpness: natural, sharp, geometric
- Background: transparent, keep, remove, solid color
- Output style: stacked shapes, cutout shapes, strokes/centerline
- Compatibility: standard SVG, Illustrator, Cricut/cutting, CAD/DXF

6.5 Advanced upscale controls

- Scale: 2x, 4x, 6x; custom dimensions later
- Model preset: auto, photo, portrait, AI art, text/shapes, transparent PNG
- Denoise: auto/low/medium/high

- Deblur: auto/low/medium/high
- Face recovery: auto/on/off
- Preserve transparency: on/off
- Output format: PNG, JPEG, WebP, TIFF

6.6 AI bot UX

The bot should support natural prompts:

- “Upscale this 4x.”
- “Make this logo into an SVG.”
- “Clean this image and remove the background.”
- “Give me a transparent PNG and SVG.”
- “Make this print-ready.”

The bot should ask at most one follow-up question when needed:

Do you want a sharper PNG, an editable SVG, or both?

The bot should return:

- Preview image
- Final download links
- Settings used
- Suggested next action

6.7 Mobile app UX

Mobile should be task-based:

- Fix my logo
- Make image HD
- Make SVG
- Remove background
- Create logo kit
- Export for print

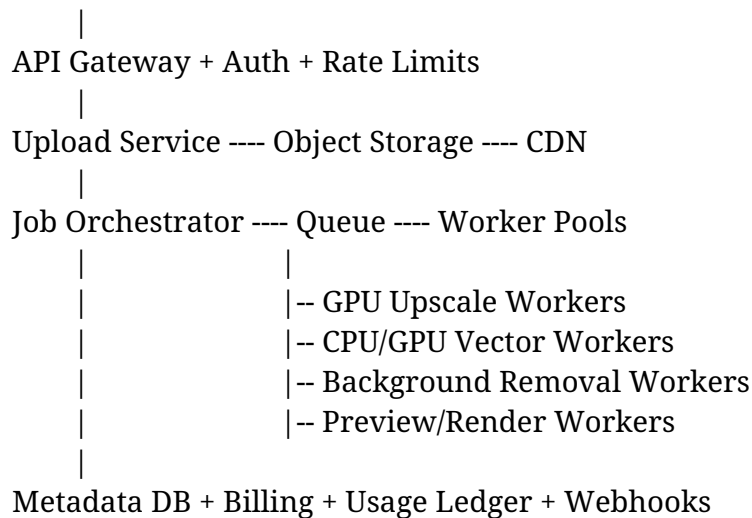
Mobile should avoid exposing many vector controls. Instead, show style cards:

- Clean logo
 - Smooth illustration
 - Sticker cutline
 - Black-and-white line art
 - Embroidery/simple colors
 - Print-ready PDF
-

7. Technical architecture

7.1 High-level architecture

Web / Mobile / Bot clients



7.2 Main services

Client apps

- Web frontend: Next.js or Remix.
- Mobile later: React Native or Flutter.
- Bot later: Poe server bot, Discord bot, Telegram bot, or WhatsApp Business API wrapper.

API gateway

Responsibilities:

- JWT/session auth.
- API key auth for developers.
- Rate limiting.
- Request validation.
- Credit balance checks.
- Signed upload URL issuance.

Upload service

Responsibilities:

- Validate file type and size.
- Strip metadata.
- Generate hash for deduplication.
- Create preview thumbnail.
- Detect dimensions, alpha, color count, file format, and likely content type.

Job orchestrator

Responsibilities:

- Create jobs.
- Pick pipeline based on image classification and user request.
- Calculate credit cost.
- Send work to queue.
- Store progress state.
- Trigger webhooks and emails.

Worker pools

Worker types:

- **Upscale GPU workers:** Real-ESRGAN, SwinIR, custom models, or provider API calls.
- **Vector workers:** VTracer, Potrace, custom contour extraction, color clustering, curve fitting, SVG optimization.
- **Background workers:** segmentation and matting.
- **Render workers:** SVG to PNG/PDF previews, thumbnails, comparison images.
- **Packaging workers:** ZIP exports, brand kits, social media sizes.

Storage

- Original uploads: private object storage.
- Intermediate files: private object storage with short TTL.
- Final exports: private storage with signed download URLs.
- Preview images: CDN with watermark or low-res limits for free users.

7.3 Suggested stack

Layer	Recommended choice	Notes
Web frontend	Next.js + TypeScript	Fast iteration, SEO landing pages, good upload UX
API	FastAPI, NestJS, or Go	Async job APIs, strong validation
Queue	Redis Queue, BullMQ, Celery, or Temporal	Use Temporal if workflows get complex
Storage	S3-compatible object storage	Signed URLs and lifecycle deletion
DB	PostgreSQL	Jobs, users, billing, usage, assets
Cache	Redis	Session cache, job progress, rate limits
GPU serving	Docker + NVIDIA runtime	Separate model containers
Vector engine	Rust/Python workers	VTracer Rust, Potrace,

Layer	Recommended choice	Notes
SVG optimization	SVGO	OpenCV, custom algorithms Optimize output SVGs [S16]
Observability	OpenTelemetry + Prometheus/Grafana	Track latency, failures, GPU use
Payments	Stripe	Credits and subscriptions

8. Image-processing pipeline design

8.1 Intake pipeline

Every upload should pass through:

1. File validation.
2. Malware and decompression-bomb guardrails.
3. EXIF and metadata stripping.
4. SHA-256 hash generation.
5. Thumbnail generation.
6. Alpha-channel detection.
7. Color histogram analysis.
8. Face/text/logo/sketch/photo classification.
9. Safe storage.
10. Recommended action generation.

8.2 Image classifier

The classifier should combine heuristics and a lightweight model.

Useful signals:

- Number of unique colors.
- Dominant palette size.
- Alpha channel.
- Edge density.
- OCR confidence.
- Face detector confidence.
- Texture complexity.
- EXIF camera metadata.
- Flat-color regions.
- Compression artifact level.

Classification labels:

- Photo
- Portrait

- AI art
- Logo/icon
- Transparent PNG
- Text/shape screenshot
- Sketch/line art
- Mixed illustration
- Low-quality scan

8.3 Upscaling pipeline

Recommended steps:

1. Normalize input color profile.
2. Detect and preserve alpha if present.
3. Tile image if too large for GPU memory.
4. Run selected model.
5. Stitch tiles with overlap blending.
6. Optional face recovery for portraits.
7. Optional denoise/deblock/deblur.
8. Optional sharpening tuned by image class.
9. Export final raster format.
10. Generate preview and compare image.

Model routing:

Image class	Model route	Notes
General photo	Photo/standard upscaler	Balanced detail and fidelity
Very low-res photo	Low-res/restoration model	Avoid plastic texture
Portrait	Upscale + face recovery	Preserve identity; avoid over-generation
AI art	Art/CG model	Preserve stylized edges
Transparent PNG	Alpha-preserving model	Preserve soft edges and transparency
Text/shapes	Conservative text/shape model	Avoid hallucinated letters
Logo	Usually vectorize; upscale only for preview or cleanup	Avoid noisy raster output

Open-source candidates:

- **Real-ESRGAN** for practical real-world image/video restoration; it supports tile options, alpha channel, grayscale, 16-bit images, and face enhancement integration through GFPGAN [S12].
- **SwinIR** for super-resolution, denoising, and JPEG artifact reduction [S13].

Provider option:

- Use Topaz API for premium upscaling during MVP if quality and launch speed matter more than unit cost [S7][S8][S9].

8.4 Vectorization pipeline

Recommended steps:

1. Crop to subject.
2. Remove background if requested.
3. Denoise and simplify colors.
4. Quantize palette.
5. Segment connected regions.
6. Extract contours.
7. Remove tiny speckles.
8. Simplify paths.
9. Detect corners.
10. Fit line, arc, ellipse, circle, rectangle, and Bezier primitives.
11. Choose stacking or cutout strategy.
12. Preserve alpha/transparency when possible.
13. Render SVG preview.
14. Optimize SVG with SVGO [S16].
15. Export SVG/PDF/EPS/DXF/PNG.

Baseline engines:

- **VTracer**: open-source raster-to-SVG converter that can vectorize JPG/PNG graphics and photographs, handle colored high-resolution scans, and expose controls such as color mode, color precision, corner threshold, speckle filtering, hierarchical strategy, curve-fitting mode, path precision, and presets [S14].
- **Potrace**: strong for black-and-white bitmap tracing, especially scanned logos, notes, and line art; it outputs SVG, PDF, EPS, PostScript, DXF, and other formats, and mkbmap can preprocess grayscale/color images [S15].

Custom improvements after MVP:

- Better palette detection.
- Shape recognition for circles, ellipses, stars, rounded rectangles, and repeated symbols.
- Text detection and optional OCR-to-font reconstruction.
- Cutline generation for stickers and vinyl.
- DXF cleanup for CNC/laser workflows.
- Adobe Illustrator and Cricut compatibility profiles.

8.5 Combined enhance + vectorize pipeline

This should be the flagship workflow.

Input logo/artwork

- > classify as logo/art/sketch/text
- > crop and remove background
- > denoise and deblock
- > conservative 2x upscale if low-res
- > edge-preserving smoothing
- > palette detection and color cleanup
- > vector tracing
- > curve fitting and corner cleanup
- > SVG optimization
- > preview and simple edit controls
- > export SVG, PNG, PDF, EPS, DXF

This pipeline is especially useful because raw tracing of a noisy low-resolution JPEG often produces hundreds or thousands of bad shapes. Cleaning first usually improves SVG quality, reduces file size, and makes paths more editable.

9. Output formats and export packages

9.1 Raster outputs

- PNG: best default for transparent graphics.
- JPEG: best for photos and smaller file sizes.
- WebP: best for web delivery.
- TIFF: pro/print tier.

9.2 Vector outputs

- SVG: default and primary vector format.
- PDF: print and client sharing.
- EPS: legacy print and design workflows.
- DXF: laser/CNC/vinyl/cutting workflows.
- PNG preview: rasterized view of vector result.

9.3 Brand/logo kit package

For logos, offer a ZIP file containing:

- logo.svg
- logo-transparent.png
- logo-white.svg
- logo-black.svg
- logo-print.pdf
- favicon-32.png
- app-icon-512.png

- social-profile-1024.png
 - brand-colors.txt
-

10. API design

The API should be async-first. Even if some jobs finish quickly, the interface should support polling, webhooks, cancellation, and later batch workflows.

10.1 Asset upload

POST /v1/assets

Request:

- Multipart image upload or source URL.
- Optional retention preference.
- Optional project ID.

Response:

```
{
  "asset_id": "ast_123",
  "width": 900,
  "height": 600,
  "format": "png",
  "has_alpha": true,
  "detected_type": "logo",
  "recommended_actions": ["enhance_vectorize", "upscale"]
}
```

10.2 Create upscale job

POST /v1/jobs/upscale

```
{
  "asset_id": "ast_123",
  "scale": 4,
  "mode": "auto",
  "preserve_alpha": true,
  "face_recovery": "auto",
  "output_format": "png"
}
```

10.3 Create vectorization job

POST /v1/jobs/vectorize

```
{
  "asset_id": "ast_123",
```

```
"mode": "logo",
"max_colors": 16,
"smoothness": 0.55,
"speckle_removal": 4,
"background": "transparent",
"outputs": ["svg", "png"]
}
```

10.4 Create combined job

POST /v1/jobs/enhance-vectorize

```
{
  "asset_id": "ast_123",
  "cleanup": "auto",
  "upscale": { "scale": 2, "model": "logo_text" },
  "vector": { "max_colors": 16, "adobe_compatible": true }
}
```

10.5 Job status

GET /v1/jobs/{job_id}

Response:

```
{
  "job_id": "job_123",
  "status": "processing",
  "progress": 72,
  "preview_url": "https://cdn.example.com/preview/job_123.png"
}
```

10.6 Download

GET /v1/jobs/{job_id}/download?format=svg

Response:

- 302 redirect to signed URL, or JSON containing short-lived signed URL.

10.7 Delete

DELETE /v1/assets/{asset_id}

DELETE /v1/jobs/{job_id}

This is important for privacy-sensitive customers and for retention-policy compliance.

11. Data model

Core tables:

users

- id, email, plan, created_at

assets

- id, user_id, original_url, preview_url
- width, height, format, has_alpha
- detected_type, sha256, retention_expires_at

jobs

- id, user_id, asset_id, job_type, status
- params_json, cost_credits, error_code
- created_at, started_at, completed_at

renditions

- id, job_id, format, url, width, height
- file_size, metadata_json, created_at

usage_ledger

- id, user_id, job_id, credits_delta, reason, created_at

api_keys

- id, user_id, hashed_key, name, scopes, last_used_at

webhooks

- id, user_id, target_url, secret, event_types, active

Later tables:

teams

projects

batch_jobs

batch_items

brand_kits

svg_edit_history

feedback_ratings

model_evaluations

12. Security, privacy, and trust

12.1 File safety

- Enforce file-size and dimension limits.
- Decode images in sandboxed workers.
- Guard against decompression bombs.

- Strip EXIF and hidden metadata by default.
- Store originals privately.
- Use signed URLs for downloads.
- Virus-scan batch ZIP uploads.

12.2 Privacy controls

Default policy:

- Delete guest uploads and results after 24 hours.
- Keep paid-user history only if the user opts in.
- Let users delete assets immediately.
- Never use customer files for model training unless explicit opt-in is given.

Enterprise options:

- Zero-retention processing.
- Private storage bucket.
- Dedicated GPU workers.
- No-training guarantee.
- Audit logs.
- SSO/SAML.

12.3 IP and terms

- Do not copy competitor outputs or train from competitor-generated results unless licensing explicitly allows it.
 - Add terms that users must have the right to upload and transform their images.
 - Add a copyright/reporting flow.
 - Watermark free previews without degrading paid exports.
-

13. Quality metrics

13.1 Upscaling quality metrics

Automated metrics:

- Perceptual quality score.
- No-reference image quality metrics.
- Face detection/identity consistency for portraits.
- Compression artifact score.
- Edge sharpness score.
- User-visible failure classifiers: plastic skin, hallucinated text, over-sharpening, ringing, tiling seams.

Human review metrics:

- Side-by-side preference tests.
- Use-case-specific ratings: photo, face, AI art, text, transparent PNG.
- Refund/download abandonment rate.

13.2 Vectorization quality metrics

Automated metrics:

- Rendered SVG vs input similarity.
- Path count.
- SVG file size.
- Number of colors.
- Speckle count.
- Palette error.
- Transparent edge quality.
- Minimum feature preservation.
- DXF import success.
- Illustrator/Cricut import success.

Human review metrics:

- “Would you download this?” preview conversion rate.
- Manual edit time saved.
- User rating after export.
- Support tickets by input type.

13.3 Golden test set

Create a private benchmark set with categories:

- Low-resolution logos.
- Transparent PNG logos.
- JPEG logos with artifacts.
- Black-and-white line art.
- Color illustrations.
- Scanned sketches.
- AI art.
- Portraits.
- Product photos.
- Text screenshots.

For each test image, store baseline outputs from internal models and manually reviewed target results.

14. Pricing and monetization

Suggested tiers

Free

- Upload and preview.
- Low-resolution watermarked raster preview.
- Limited daily jobs.
- No full SVG download.

Pay-as-you-go

- Credits for full exports.
- Good for occasional logo users.

Pro subscription

- Higher monthly credits.
- SVG/PDF/EPS/DXF exports.
- Batch processing.
- Brand kits.
- Priority queue.

Business/API

- API keys.
- Webhooks.
- Team users.
- Higher file limits.
- Commercial usage.
- White-label widget.

Enterprise

- Dedicated workers.
- Zero retention.
- SSO.
- SLA.
- Custom workflows.

Credit model

Possible credit pricing:

- Vector preview: free or low-cost.
- Full SVG download: 1 credit.
- 2x upscale: 1 credit per output megapixel band.
- 4x upscale: 2-4 credits depending on output size.
- Enhance + vectorize: 2 credits.

- Batch/API: discounted by volume.

The credit model should be easy to understand. Users dislike complex hidden pricing. Show estimated credits before running full jobs.

15. MVP scope

MVP must have

- Web upload.
- Guest preview.
- Account creation.
- Auto image classification.
- 2x and 4x upscaling.
- SVG vectorization for logos and simple illustrations.
- Transparent PNG preservation.
- Before/after preview.
- SVG raster preview.
- Download PNG and SVG.
- Basic payment/credits.
- Basic job queue and status polling.
- Basic deletion controls.

MVP should not include yet

- Full vector editor.
- Perfect text-to-font reconstruction.
- Full mobile app.
- Enterprise SSO.
- Realtime collaborative editing.
- Complex CAD cleanup.
- Training custom models per customer.

MVP success criteria

- User can upload a poor-quality logo and download a useful SVG in under a few minutes.
 - User can upscale a photo or AI image to 4x and see clear improvement.
 - Free preview convinces enough users to pay for full export.
 - Support volume remains manageable.
 - Processing cost per paid job is below target margin.
-

16. Roadmap

Phase 0 - Prototype

Goal: prove technical feasibility.

Deliverables:

- Upload page.
- Run Real-ESRGAN or provider upscaling.
- Run VTracer/Potrace vectorization.
- Export SVG and PNG.
- Manual settings panel.
- Internal benchmark set.

Phase 1 - MVP beta

Goal: launch to early users.

Deliverables:

- Auto mode.
- Preview UI.
- Queue and status polling.
- Credit system.
- Stripe checkout.
- Delete files.
- Basic analytics.
- Error handling and retries.

Phase 2 - Public v1

Goal: monetize and improve quality.

Deliverables:

- Better logo pipeline.
- Background removal.
- Palette editor.
- Batch upload.
- Brand kit ZIP.
- API beta.
- Webhooks.
- User feedback loop.

Phase 3 - Pro workflows

Goal: serve designers and print shops.

Deliverables:

- PDF/EPS/DXF exports.
- Illustrator/Cricut compatibility modes.
- Cutline generation.
- White-label widget.
- Team accounts.
- Higher file limits.

Phase 4 - Mobile and bot

Goal: expand channels.

Deliverables:

- Mobile app with simple task cards.
- Poe/Discord/Telegram bot wrapper.
- Camera scan workflow.
- Mobile brand-kit export.

Phase 5 - Proprietary quality moat

Goal: improve beyond open-source baselines.

Deliverables:

- Custom vector graph representation.
 - Shape recognition.
 - Better curve/corner modeling.
 - SVG edit suggestions.
 - Custom upscaling model fine-tuned for logos/text.
 - Quality-prediction model for preview confidence.
-

17. Team and effort estimate

Lean MVP team

- 1 product/design lead.
- 1 frontend engineer.
- 1 backend engineer.
- 1 ML/image-processing engineer.
- 1 part-time DevOps engineer.
- 1 part-time QA/support person.

Estimated MVP timeline

Workstream	Duration	Notes
Technical prototype	2-4 weeks	Baseline

Workstream	Duration	Notes
Web app MVP	4-6 weeks	upscaling/vectorization Upload, preview, download, account
Backend/API/job queue	4-6 weeks	Can run parallel with frontend
Payments and credits	1-2 weeks	Stripe integration
QA and benchmark set	ongoing	Needs real user files
Beta hardening	2-4 weeks	Error handling, retries, cost controls

A realistic first paid beta is possible in **8-12 weeks** with a focused team and careful scope control.

18. Major risks and mitigations

Risk 1: Vectorization quality is inconsistent

Mitigation:

- Start with logos, icons, sketches, and simple art.
- Show preview before charging.
- Add input-type warnings for unsuitable photos.
- Build feedback collection into the preview screen.

Risk 2: Upscaling can hallucinate details

Mitigation:

- Use conservative models for text/logos.
- Label generative restoration clearly.
- Add side-by-side preview.
- Preserve original download and settings.

Risk 3: GPU costs become too high

Mitigation:

- Use tiled inference.
- Queue jobs by priority.
- Use CPU vectorization where possible.
- Cache duplicate uploads by hash.
- Offer provider fallback only for premium jobs.

Risk 4: SVG output is not compatible with user software

Mitigation:

- Test with Illustrator, Inkscape, Cricut Design Space, laser/CNC software, and common print workflows.
- Offer compatibility presets.
- Include rasterized PNG preview in every vector ZIP.

Risk 5: Users upload copyrighted or sensitive files

Mitigation:

- Add clear terms.
 - Default short retention.
 - No-training default.
 - User deletion controls.
 - Abuse reporting and takedown process.
-

19. Recommended build decision

Best initial product

Build a **web app** with an API-first backend.

Reasoning:

- Users often work with files on desktop.
- SVG inspection and editing are easier on web.
- Web supports drag/drop, ZIP downloads, and payment conversion better than mobile.
- API-first architecture makes it easier to add mobile and bot clients later.

Best technical path

Use a hybrid model strategy:

1. **MVP**: combine open-source engines and/or provider APIs.
2. **V1**: improve routing, preprocessing, and SVG cleanup.
3. **V2**: build proprietary vector geometry and model improvements.

Best market wedge

Start with:

Fix my logo and make it SVG.

This is a clearer buyer intent than broad photo upscaling. Users with logos often have immediate commercial needs: printing, websites, packaging, stickers, embroidery, or signage.

20. Final recommended feature list

Launch features

- Upload image.
- Auto-detect best action.
- Upscale 2x/4x.
- Vectorize to SVG.
- Enhance + vectorize mode.
- Remove background.
- Preserve transparency.
- Before/after preview.
- Download SVG and PNG.
- Basic credits/payment.
- Delete uploaded files.

Fast-follow features

- PDF/EPS/DXF export.
- Batch processing.
- Brand kit ZIP.
- API keys.
- Webhooks.
- Palette editor.
- Speckle removal slider.
- Cricut/Illustrator presets.

Long-term moat features

- Proprietary vector graph.
 - Shape recognition.
 - Text reconstruction.
 - Cutline generation.
 - Custom logo/text upscaler.
 - On-device mobile preview.
 - Enterprise privacy mode.
-

21. Source notes

[S1] Vectorizer.io homepage, public feature description: <https://www.vectorizer.io/>

[S2] Vectorizer.io API documentation, version 4.0.x: <https://www.vectorizer.io/api/>

[S3] Vectorizer.AI homepage and FAQ: <https://vectorizer.ai/>

[S4] Vectorizer.AI API documentation: <https://vectorizer.ai/api>

[S5] Topaz Gigapixel product page: <https://www.topazlabs.com/gigapixel>

[S6] Topaz Gigapixel upscale/resize documentation:
<https://docs.topazlabs.com/gigapixel-ai/filters-panel/resize-mode>
[S7] Topaz Developer Documentation - Gigapixel image models:
<https://developer.topazlabs.com/image-models/gigapixel>
[S8] Topaz Image API Tool async endpoint:
<https://developer.topazlabs.com/api-reference/image-api/tool>
[S9] Topaz Image API Status endpoint: <https://developer.topazlabs.com/api-reference/api-endpoints/image/status>
[S10] Poe TopazLabs bot listing: <https://poe.com/TopazLabs>
[S11] AppBrain listing for AI Logo Maker / com.logomaker.logo.creator:
<https://www.appbrain.com/app/ai-logo-maker/com.logomaker.logo.creator>
[S12] Real-ESRGAN official GitHub repository: <https://github.com/xinntao/Real-ESRGAN>
[S13] SwinIR official GitHub repository: <https://github.com/JingyunLiang/SwinIR>
[S14] VTracer official GitHub repository: <https://github.com/visioncortex/vtracer>
[S15] Potrace official site: <https://potrace.sourceforge.net/>
[S16] SVGGO official documentation: <https://svggo.dev/>

22. Appendix: implementation checklist

Backend checklist

- ☐ Asset upload endpoint.
- ☐ Signed upload and download URLs.
- ☐ Image metadata extraction.
- ☐ Image classification router.
- ☐ Async job queue.
- ☐ Upscale worker.
- ☐ Vector worker.
- ☐ Preview renderer.
- ☐ Usage ledger.
- ☐ Billing integration.
- ☐ Retention deletion job.
- ☐ API key management.
- ☐ Webhook sender.
- ☐ Monitoring dashboard.

Frontend checklist

- ☐ Landing page.
- ☐ Drag/drop uploader.
- ☐ Image preview.
- ☐ Auto recommendation UI.
- ☐ Before/after slider.
- ☐ Vector preview toggle.
- ☐ Advanced settings drawer.

- ☐ Download/paywall modal.
- ☐ Account dashboard.
- ☐ Job history.
- ☐ Deletion controls.

ML/image checklist

- ☐ Benchmark set.
- ☐ Real-ESRGAN baseline.
- ☐ SwinIR baseline.
- ☐ VTracer baseline.
- ☐ Potrace baseline.
- ☐ Background removal baseline.
- ☐ SVG optimizer.
- ☐ Render-diff evaluation.
- ☐ Human rating UI.
- ☐ Failure taxonomy.